

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 17
“SKEMA PEMROSESAN SEKUENSIAL”



DISUSUN OLEH:
Alvin Setya Wardana
109082500107
S1 IF-13-02
DOSEN:
WAHYU ANDI SAPUTRA

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

DASAR TEORI

1. Latihan1 Source

Code:

```
package main

import (
    "fmt"
)

func main() {
    var x float64
    var jumlah float64
    var hitung int

    fmt.Println("Masukkan bilangan real (akhiri dengan 9999):")
    for {
        fmt.Scan(&x)
        if x == 9999 {
            break
        }
        jumlah += x
        hitung++
    }

    if hitung > 0 {
        rata := jumlah / float64(hitung)
        fmt.Printf("Rata-rata = %.2f\n", rata)
    } else {
        fmt.Println("Tidak ada data yang dihitung.")
    }
}
```

Output:

The screenshot shows the Visual Studio Code interface with a Go file named `package main.go` open. The code is as follows:

```
14 func main() {
15     fmt.Scan(&x)
16     if x == 9999 {
17         break
18     }
19     jumlah += x
20     hitung++
21 }
22 if hitung > 0 {
23     rata := jumlah / float64(hitung)
24     fmt.Printf("Rata-rata = %.2f\n", rata)
25 } else {
26     fmt.Println("Tidak ada data yang dihitung.")
27 }
28 }
29 }
```

The terminal at the bottom shows the command `go run "d:\telyu\alpro_praktek\modul14\package main.go"` being executed. The output is:

```
PS C:\Users\LENOVO> go run "d:\telyu\alpro_praktek\modul14\package main.go"
Masukkan bilangan real (akhiri dengan 9999):
93
92
95
99
9999
Rata-rata = 94.75
PS C:\Users\LENOVO>
```

A notification at the bottom right indicates that the R package (languageserver) is required to enable R language service features.

Deskripsi Program:

Program tersebut membaca bilangan real dari input hingga menemukan angka **9999** sebagai penanda berhenti, lalu menghitung jumlah dan banyak data yang dimasukkan. Setelah itu, jika ada data, program mencetak rata-rata; jika tidak ada, program menampilkan pesan bahwa tidak ada data yang dihitung.

Latihan 2

Source Code:

```
package main

import (
    "fmt"
)

func main() {
    var x string
    var n int

    fmt.Print("Masukkan string x: ")
    fmt.Scan(&x)

    fmt.Print("Masukkan jumlah data string (n): ")
    fmt.Scan(&n)

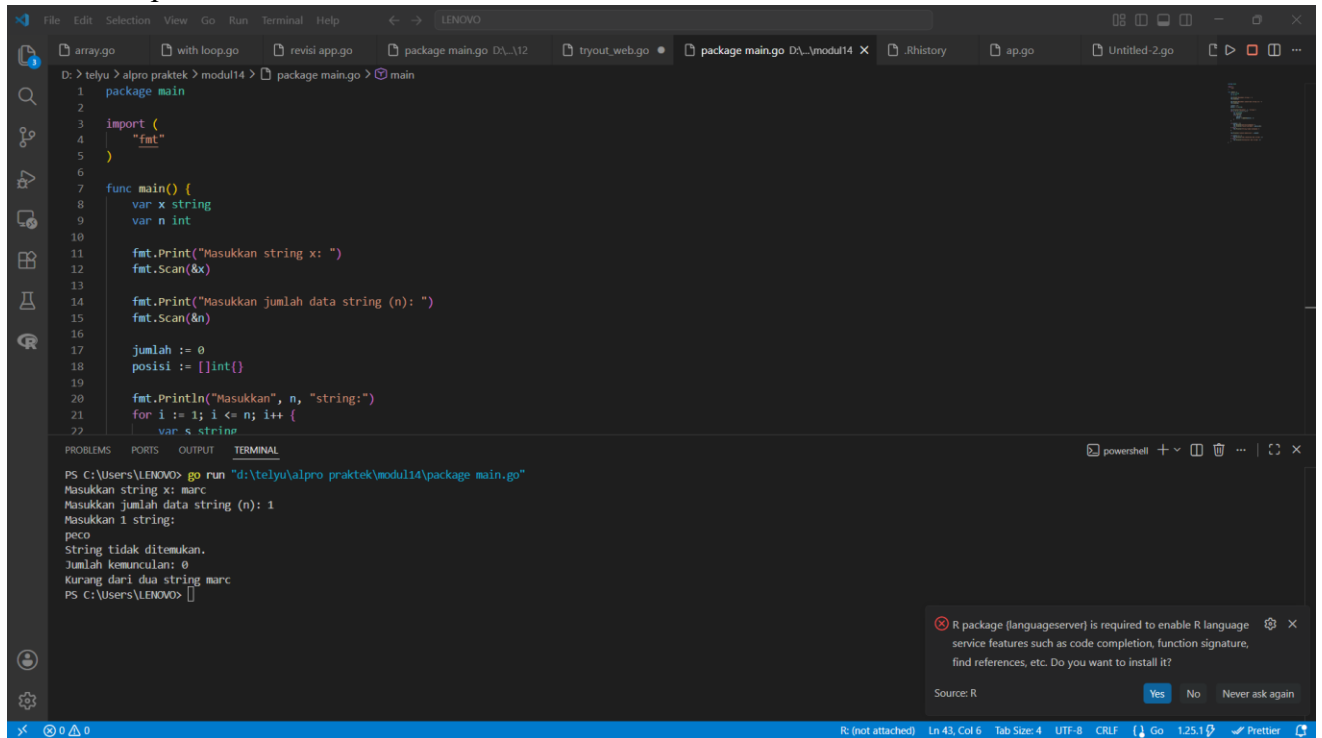
    jumlah := 0
    posisi := []int{}

    fmt.Println("Masukkan", n, "string:")
    for i := 1; i <= n; i++ {
        var s string
        fmt.Scan(&s)
        if s == x {
            jumlah++
            posisi = append(posisi, i)
        }
    }

    if jumlah > 0 {
        fmt.Println("String ditemukan.")
        fmt.Println("Posisi pertama:", posisi[0])
    } else {
        fmt.Println("String tidak ditemukan.")
    }

    fmt.Println("Jumlah kemunculan:", jumlah)
    if jumlah >= 2 {
        fmt.Println("Ada sedikitnya dua string", x)
    } else {
        fmt.Println("Kurang dari dua string", x)
    }
}
```

Output:



The screenshot shows a Visual Studio Code editor window with a Go file named `package main.go` open. The code defines a `main` function that prompts the user for a string `x` and an integer `n`. It then checks if the string `x` appears `n` times in the input. The terminal output shows the program running successfully with the input `marc` and `1`, resulting in the message `String tidak ditemukan.` (String not found).

```
1 package main
2
3 import (
4     "fmt"
5 )
6
7 func main() {
8     var x string
9     var n int
10
11     fmt.Print("Masukkan string x: ")
12     fmt.Scan(&x)
13
14     fmt.Print("Masukkan jumlah data string (n): ")
15     fmt.Scan(&n)
16
17     jumlah := 0
18     posisi := []int{}
19
20     fmt.Println("Masukkan", n, "string:")
21     for i := 1; i <= n; i++ {
22         var s string
```

Terminal Output:

```
PS C:\Users\LENOVO> go run "d:\telyu\alpro praktik\modul14\package main.go"
Masukkan string x: marc
Masukkan jumlah data string (n): 1
Masukkan 1 string:
peco
String tidak ditemukan.
Jumlah kemunculan: 0
Kurang dari dua string marc
PS C:\Users\LENOVO>
```

Deskripsi Program:

Program tersebut membaca sebuah string `x` dan jumlah data `n`, lalu meminta pengguna memasukkan `n` string untuk diperiksa. Program menghitung berapa kali string `x` muncul, mencatat posisi kemunculan pertama, kemudian menampilkan apakah string ditemukan, jumlah kemunculan, serta apakah ada sedikitnya dua kali kemunculan.

Latihan 3

Source Code:

```
package main
import (
    "fmt"
    "math/rand"
    "time"
)
func main() {
    var n int
    fmt.Print("Masukkan jumlah tetesan hujan: ")
    fmt.Scan(&n)

    rand.Seed(time.Now().UnixNano())

    countA, countB, countC, countD := 0, 0, 0, 0

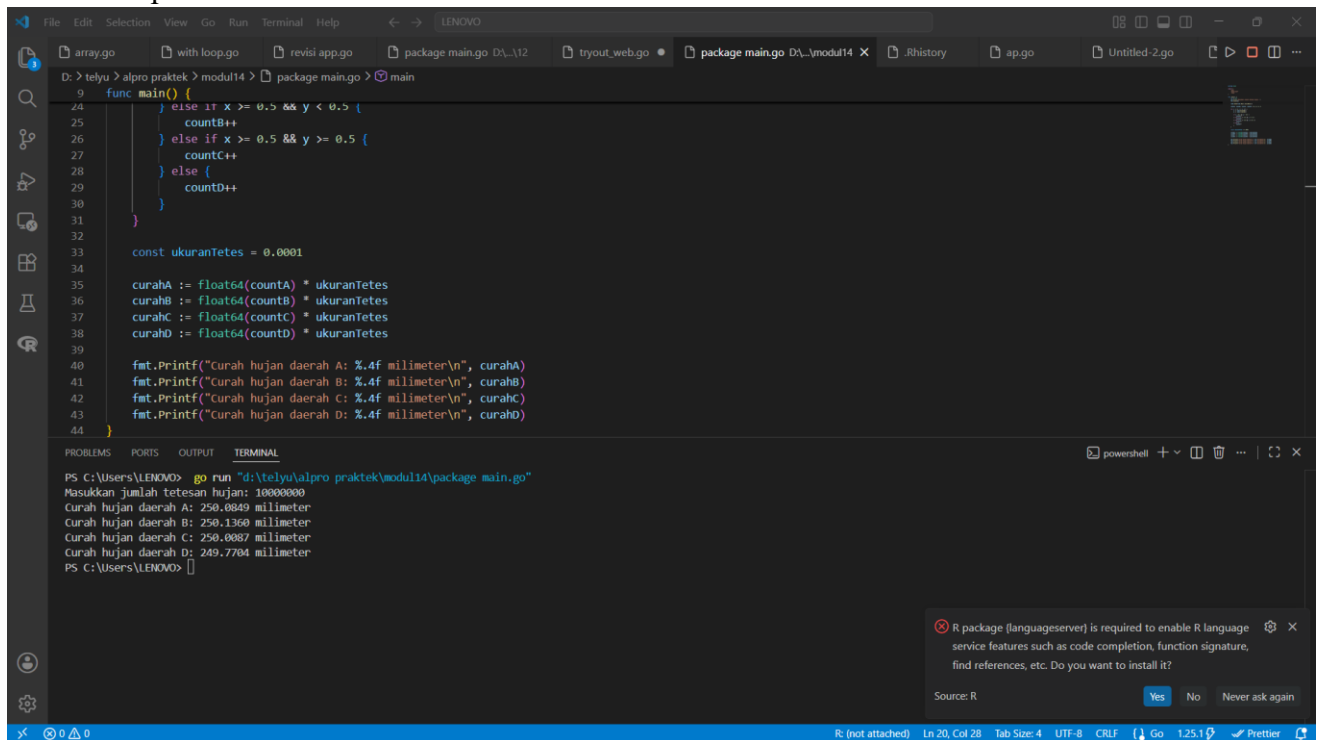
    for i := 0; i < n; i++ {
        x := rand.Float64() // koordinat x [0..1]
        y := rand.Float64() // koordinat y [0..1]

        if x < 0.5 && y < 0.5 {
            countA++
        } else if x >= 0.5 && y < 0.5 {
            countB++
        } else if x >= 0.5 && y >= 0.5 {
            countC++
        } else {
            countD++
        }
    }

    const ukuranTetes = 0.0001
    curahA := float64(countA) * ukuranTetes
    curahB := float64(countB) * ukuranTetes
    curahC := float64(countC) * ukuranTetes
    curahD := float64(countD) * ukuranTetes

    fmt.Printf("Curah hujan daerah A: %.4f milimeter\n", curahA)
    fmt.Printf("Curah hujan daerah B: %.4f milimeter\n", curahB)
    fmt.Printf("Curah hujan daerah C: %.4f milimeter\n", curahC)
    fmt.Printf("Curah hujan daerah D: %.4f milimeter\n", curahD)
}
```

Output:



The screenshot shows a Visual Studio Code editor with a Go file named `main.go` and a terminal window. The Go code defines a `main` function that simulates rain distribution. It uses a random number generator to place 1,000,000 raindrops into four quadrants (A, B, C, D) based on their coordinates. The counts for each quadrant are then multiplied by a constant `ukuranTetes = 0.0001` to calculate the rainfall in millimeters. The results are printed to the console.

```
9 func main() {
24     } else if x >= 0.5 && y < 0.5 {
25         countB++
26     } else if x >= 0.5 && y >= 0.5 {
27         countC++
28     } else {
29         countD++
30     }
31 }
32
33 const ukuranTetes = 0.0001
34
35 curahA := float64(countA) * ukuranTetes
36 curahB := float64(countB) * ukuranTetes
37 curahC := float64(countC) * ukuranTetes
38 curahD := float64(countD) * ukuranTetes
39
40 fmt.Printf("curah hujan daerah A: %.4f millimeter\n", curahA)
41 fmt.Printf("curah hujan daerah B: %.4f millimeter\n", curahB)
42 fmt.Printf("curah hujan daerah C: %.4f millimeter\n", curahC)
43 fmt.Printf("curah hujan daerah D: %.4f millimeter\n", curahD)
44 }
```

The terminal output shows the execution of the program:

```
PS C:\Users\LENOVO> go run "d:\telyu\alpro praktek\modul14\package main.go"
Masukkan jumlah tetesan hujan: 10000000
curah hujan daerah A: 250.0849 millimeter
curah hujan daerah B: 250.1360 millimeter
curah hujan daerah C: 250.0887 millimeter
curah hujan daerah D: 249.7704 millimeter
PS C:\Users\LENOVO>
```

A notification at the bottom right indicates that the R package `(languageserver)` is required to enable R language service features such as code completion, function signature, and find references, etc. The user is prompted to install it, with options: Yes, No, or Never ask again.

Deskripsi Program:

Program tersebut melakukan simulasi curah hujan dengan cara menghasilkan koordinat acak (x,y) untuk setiap tetesan hujan, lalu menghitung jumlah tetesan yang jatuh di empat daerah berbeda (A, B, C, D) berdasarkan posisi kuadran. Setelah jumlah tetesan dihitung, program mengonversinya menjadi curah hujan dalam milimeter dengan rumus $\text{jumlah_tetesan} \times 0.0001$ dan menampilkan hasil untuk masing-masing daerah.

Source Code:

4

```
package main

import (
    "fmt"
    "math"
)

func main() {
    var N int
    fmt.Print("N suku pertama: ")
    fmt.Scan(&N)

    sum := 0.0
    for i := 1; i <= N; i++ {
        term := math.Pow(-1, float64(i+1)) / float64(2*i-1)
        sum += term
    }

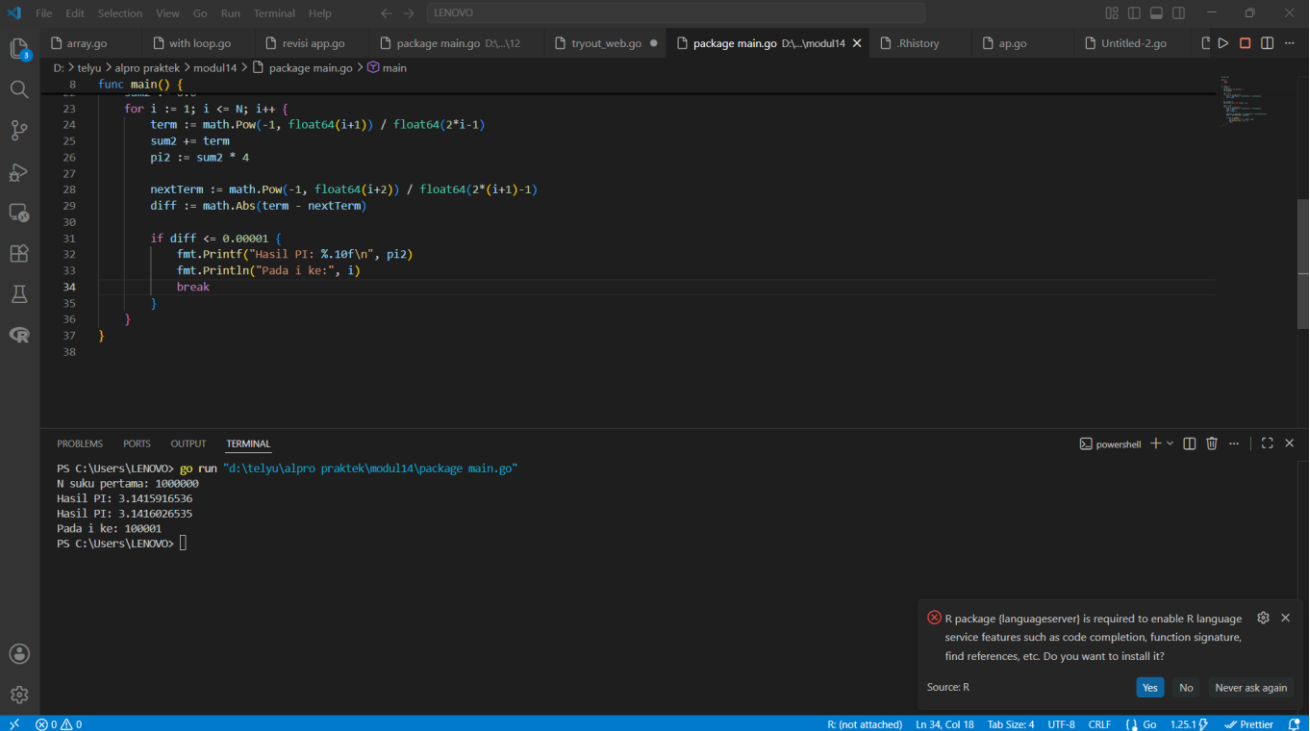
    pi := sum * 4
    fmt.Printf("Hasil PI: %.10f\n", pi)

    // Modifikasi: berhenti jika selisih dua suku bersebelahan <= 0.00001
    sum2 := 0.0
    for i := 1; i <= N; i++ {
        term := math.Pow(-1, float64(i+1)) / float64(2*i-1)
        sum2 += term
        pi2 := sum2 * 4

        // hitung selisih dengan suku berikutnya
        nextTerm := math.Pow(-1, float64(i+2)) / float64(2*(i+1)-1)
        diff := math.Abs(term - nextTerm)

        if diff <= 0.00001 {
            fmt.Printf("Hasil PI: %.10f\n", pi2)
            fmt.Println("Pada i ke:", i)
            break
        }
    }
}
```

Output:



```
8 func main() {
23     for i := 1; i <= N; i++ {
24         term := math.Pow(-1, float64(i+1)) / float64(2*i-1)
25         sum2 += term
26         pi2 := sum2 * 4
27
28         nextTerm := math.Pow(-1, float64(i+2)) / float64(2*(i+1)-1)
29         diff := math.Abs(term - nextTerm)
30
31         if diff <= 0.00001 {
32             fmt.Printf("Hasil PI: %.10f\n", pi2)
33             fmt.Println("pada i ke:", i)
34             break
35         }
36     }
37 }
38 }
```

```
PS C:\Users\LENOVO> go run "d:\telyu\alpro praktik\modul14\package main.go"
N suku pertama: 1000000
Hasil PI: 3.1415916536
Hasil PI: 3.1416026535
pada i ke: 100001
PS C:\Users\LENOVO>
```

R package (languageserver) is required to enable R language service features such as code completion, function signature, find references, etc. Do you want to install it?

Source: R

Yes No Never ask again

deskripsi

Program tersebut menghitung pendekatan nilai π menggunakan deret Leibniz, di mana setiap suku dihitung dengan rumus $(-1)^{i+1}/(2i-1)$ lalu dijumlahkan dan dikalikan 4. Versi modifikasi akan berhenti otomatis jika selisih dua suku berurutan lebih kecil atau sama dengan 0.00001, sehingga efisien dalam menentukan titik konvergensi.

Source code

5

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

func main() {
    var n int
    fmt.Print("Banyak Topping: ")
    fmt.Scan(&n)

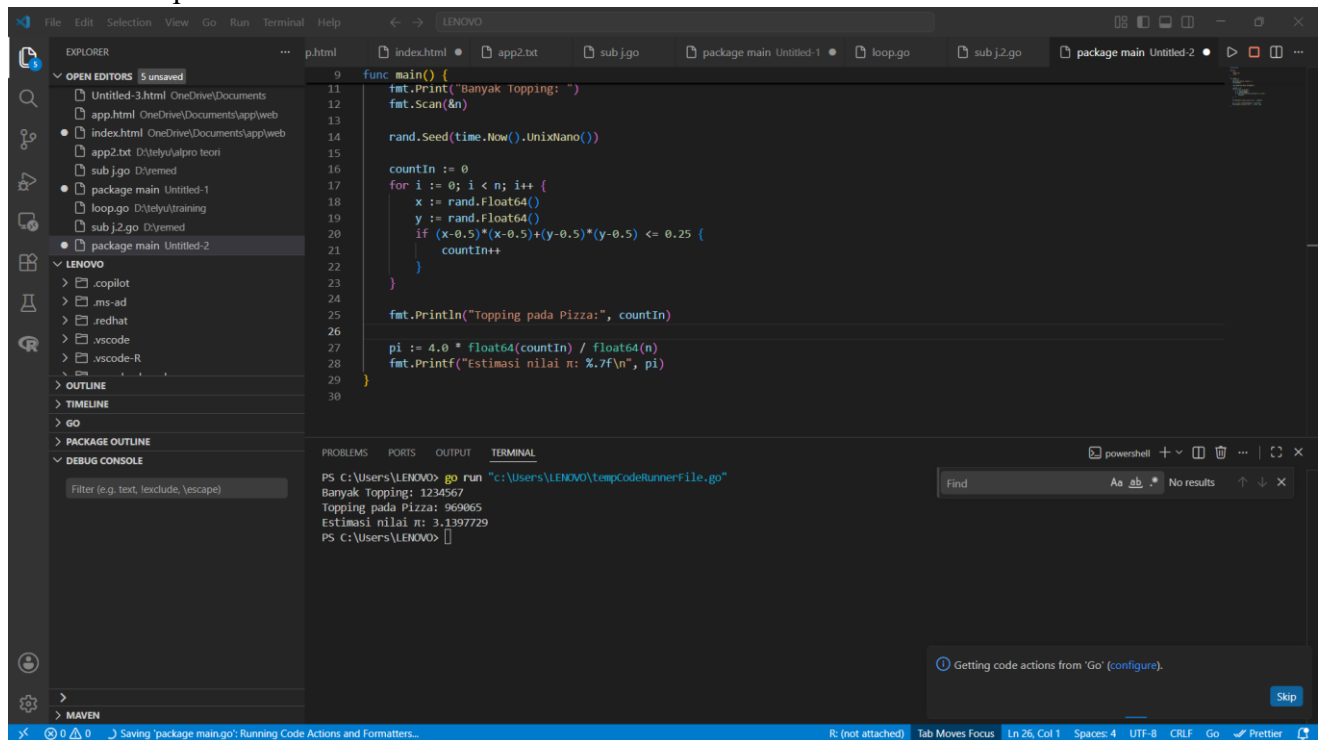
    rand.Seed(time.Now().UnixNano())

    countIn := 0
    for i := 0; i < n; i++ {
        x := rand.Float64()
        y := rand.Float64()
        if (x-0.5)*(x-0.5)+(y-0.5)*(y-0.5) <= 0.25 {
            countIn++
        }
    }

    fmt.Println("Topping pada Pizza:", countIn)

    pi := 4.0 * float64(countIn) / float64(n)
    fmt.Printf("Estimasi nilai  $\pi$ : %.7f\n", pi)
}
```

Output:



```
func main() {
    fmt.Println("Banyak Topping: ")
    fmt.Scan(&n)

    rand.Seed(time.Now().UnixNano())

    countIn := 0
    for i := 0; i < n; i++ {
        x := rand.Float64()
        y := rand.Float64()
        if (x-0.5)*(x-0.5)+(y-0.5)*(y-0.5) <= 0.25 {
            countIn++
        }
    }

    fmt.Println("Topping pada Pizza:", countIn)

    pi := 4.0 * float64(countIn) / float64(n)
    fmt.Printf("Estimasi nilai pi: %.7f\n", pi)
}
```

PS C:\Users\LENOVO> go run "c:\Users\LENOVO\tempCodeRunnerFile.go"

Banyak Topping: 1234567

Topping pada Pizza: 969065

Estimasi nilai pi: 3.1397729

PS C:\Users\LENOVO> |

Deskripsi program:

Program simulasi Monte Carlo ini menaburkan titik acak sebagai topping pada wadah berbentuk persegi, lalu menghitung berapa banyak yang jatuh di dalam lingkaran pizza dengan pusat (0.5, 0.5) dan jari-jari 0.5. Rasio jumlah topping di dalam pizza terhadap total topping digunakan untuk memperkirakan nilai konstanta π .

DAFTAR PUSTAKA

- Dasa, Rinaldi Munir. *Modul Praktikum Algoritma dan Pemrograman 2*. Telkom University, 2024.
- Cormen, Thomas H., et al. *Introduction to Algorithms*. MIT Press, 2009.
- Wirth, Niklaus. *Algorithms + Data Structures = Programs*. Prentice Hall, 1976.
- Knuth, Donald E. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Addison-Wesley, 1997.
- Sedgewick, Robert, dan Wayne, Kevin. *Algorithms*. Addison-Wesley, 2011.